DIGITAL NOTES

ON

CLOUD COMPUTING

III B.TECH – II SEM

Prepared by

Dr. T V DIVYA

Asst Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Sreyas Institute of Engineering and Technology

An Autonomous Institution

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC-A Grade, NBA (CSE, ECE & ME) & ISO 9001:2015 Certified

Bandlaguda, Nagole, Hyderabad, Telangana 500068

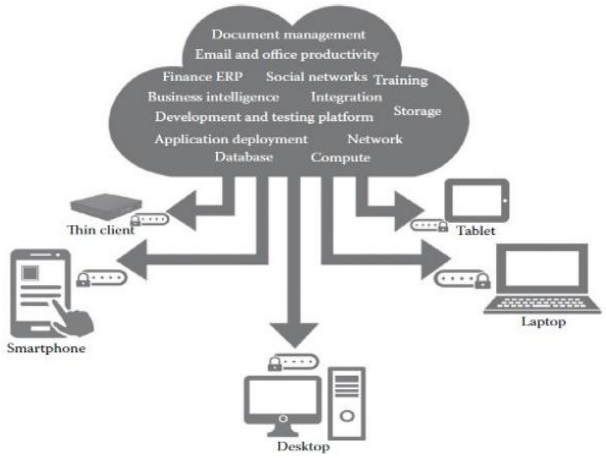
UNIT II

Cloud computing fundamentals: Motivation for Cloud Computing, The Need for Cloud Computing, Definition of Cloud computing, Principles of Cloud computing, Five Essential Characteristics, Four Cloud Deployment Models, on demand services like Elastic resource pooling using Amazon Elastic Compute Cloud (EC2) as example, Rapid elasticity using Amazon EBS, Amazon EFS, Amazon S3, Amazon LEX, Amazon Lambda, overview of Docker CLI commands cloud deployment using Docker.

MOTIVATION CLOUD COMPUTING

- The users who are in need of computing are expected to invest more money on computing resources such as hardware, software, networking, and storage; this investment naturally costs a bulk currency to buy these computing resources, all these tasks would add cost huge expenditure to the classical academics and individuals.
- On the other hand, it is easy and handy to get the required computing power and resources from some provider (or supplier) as and when it is needed and pay only for that usage. This would cost only a reasonable investment or spending, compared to the huge investment when buying the entire computing infrastructure. This phenomenon can be viewed as *capital expenditure* versus *operational expenditure*.
- As one can easily assess the huge lump sum or smaller lump sum required for the hiring or getting the computing infrastructure only to the tune of required time, and rest of the time free from that.
- Cloud computing is a mechanism of bringing–hiring or getting the services of the computing power or infrastructure to an organizational or individual level to the extent required and paying only for the consumed services.

  Example : Electricity in our homes or offices
- Cloud computing is very Economical and saves a lot of money.
- A blind benefit of this computing is that even if we lose our laptop or due to some crisis our personal computer—and the desktop system—gets damaged, still our data and files will stay safe and secured as these are not in our local machine (but remotely located at the provider's place—machine).
- It is a fast solution growing in popularity because of storage especially among individuals and small and medium-sized companies (SMEs).
- Thus, cloud computing comes into focus and much needed subscription based or pay-per-use service model of offering computing to end users or customers over the Internet and thereby extending the IT's existing capabilities.



NEED FOR CLOUD COMPUTING

- The main reasons for the need and use of cloud computing are convenience and reliability.
- In the past, if we wanted to bring a file, we would have to save it to a Universal Serial Bus (USB) flash drive, external hard drive, or compact disc (CD) and bring that device to a different place.
- Instead, saving a file to the cloud (e.g., use of cloud application Dropbox) ensures that we will be able to access it with any computer that has an Internet connection. The cloud also makes it much easier to share a file with friends, making it possible to collaborate over the web.
- While using the cloud, losing our data/file is much less likely. However, just like anything online, there is always a risk that someone may try to gain access to our personal data, and therefore, it is important to choose an access control with a strong password and pay attention to any privacy settings for the cloud service that we are using.

NIST DEFINITION OF CLOUD COMPUTING

The formal definition of cloud computing comes from the National Institute of Standards and Technology (NIST): "Cloud computing is a model for enabling ubiquitous, convenient, on demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.

PRINCIPLES OF CLOUD COMPUTING

The 5-4-3 principles put forth by NIST describe :

(a) Five essential characteristic features that promote cloud computing

(b) Four deployment models that are used to narrate the cloud computing opportunities for Customers while looking at architectural models.

(c) Three important and basic service offering models of cloud computing

(a)Five Essential Characteristics

Cloud computing has five essential characteristics, which are shown in Figure 2.2. Readers can note the word *essential*, which means that if any of these characteristics is missing, then it is not cloud computing:

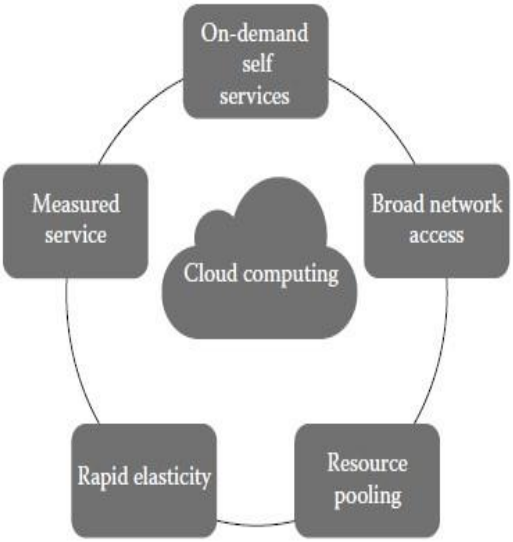


FIGURE 2.2
The essential characteristics of cloud computing.

1. **On-demand self-service:** A consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with each service's provider.
2. **Broad network access:** Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and personal digital assistants [PDAs])
3. **Elastic resource pooling:** The provider's computing resources are pooled to serve multiple consumers using a multitenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There is a sense of location independence in that the customer generally has no control or knowledge over the exact location of the provided resources but may be able to specify the location at a higher level of abstraction (e.g., country, state, or data center). Examples of resources include storage, processing, memory, and network bandwidth.
4. **Rapid elasticity:** Capabilities can be rapidly and elastically provisioned, in some cases automatically, to quickly scale out and rapidly released to quickly scale in. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time.
5. **Measured service:** Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing , bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported providing transparency for both the provider and consumer of the utilized service.

(b) Four Cloud Deployment Models

Deployment models also called as types of models. These deployment models describe the ways with which the cloud services can be deployed or made available to its customers, depending on the organizational structure and the provisioning location. One can understand it in this manner too: cloud (Internet)-based computing resources—that is, the locations where data and services are acquired and provisioned to its customers—can take various forms. Four deployment models are usually distinguished, namely,

(1)Public

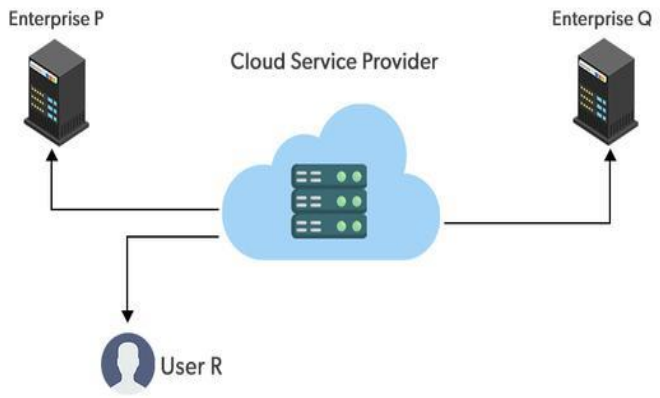
(2)Private

(3)Community, and

(4)Hybrid cloud service usage

(1)Public Cloud

- Public clouds are managed by third parties which provide cloud services over the internet to the public, these services are available as pay-as-you-go billing models.
- They offer solutions for minimizing IT infrastructure costs and become a good option for handling peak loads on the local infrastructure.
- Public clouds are the go-to option for small enterprises, which can start their businesses without large upfront investments by completely relying on public infrastructure for their IT needs.
- The fundamental characteristics of public clouds are **multitenancy**. A public cloud is meant to serve multiple users, not a single customer. A user requires a virtual computing environment that is separated, and most likely isolated, from other users.



Advantages of using a Public cloud are:

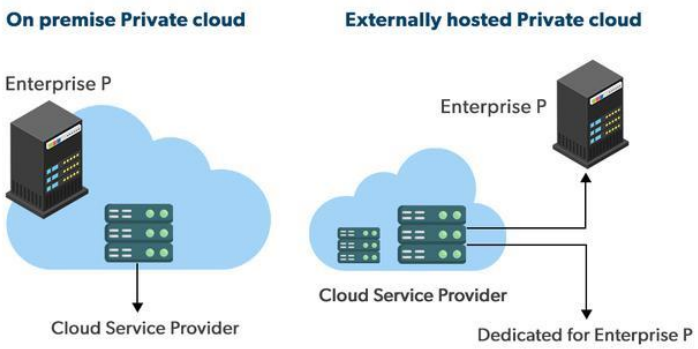
1. High Scalability
2. Cost Reduction
3. Reliability and flexibility
4. Disaster Recovery

Disadvantages of using a Public cloud are:

1. Loss of control over data
2. Data security and privacy
3. Limited Visibility
4. Unpredictable cost

(2)Private cloud

Private clouds are distributed systems that work on private infrastructure and provide the users with dynamic provisioning of computing resources. Instead of a pay-as-you-go model in private clouds, there could be other schemes that manage the usage of the cloud and proportionally billing of the different departments or sections of an enterprise. Private cloud providers are HP Data Centers, Ubuntu, Elastic-Private cloud, Microsoft, etc.



Private Cloud

Advantages of using a private cloud are as follows:

1. **Customer information protection:** In the private cloud security concerns are less since customer data and other sensitive information do not flow out of private infrastructure.
2. **Infrastructure ensuring SLAs:** Private cloud provides specific operations such as appropriate clustering, data replication, system monitoring, and maintenance, disaster recovery, and other uptime services.
3. **Compliance with standard procedures and operations:** Specific procedures have to be put in place when deploying and executing applications according to third-party compliance standards. This is not possible in the case of the public cloud.

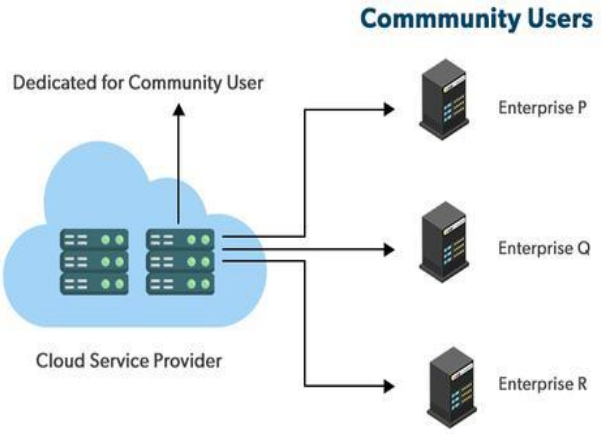
Disadvantages of using a private cloud are:

1. **The restricted area of operations:** Private cloud is accessible within a particular area. So the area of accessibility is restricted.

(3)Community cloud

Community clouds are distributed systems created by integrating the services of different clouds to address the specific needs of an industry, a community, or a business sector. But sharing responsibilities among the organizations is difficult.

In the community cloud, the infrastructure is shared between organizations that have shared concerns or tasks. An organization or a third party may manage the cloud.



Community Cloud

Advantages of using Community cloud are:

1. Because the entire cloud is shared by numerous enterprises or a community, community clouds are cost-effective.
2. Because it works with every user, the community cloud is adaptable and scalable. Users can alter the documents according to their needs and requirements.
3. Public cloud is less secure than the community cloud, which is more secure than private cloud.
4. Thanks to community clouds, we may share cloud resources, infrastructure, and other capabilities between different enterprises.

Disadvantages of using Community cloud are:

1. Not all businesses should choose community cloud.
2. gradual adoption of data
3. It's challenging for corporations to share duties.

Sectors that use community clouds are:

1. Media industry: Media companies are looking for quick, simple, low-cost ways for increasing the efficiency of content generation. Most media productions involve an extended ecosystem of partners. In particular, the creation of digital content is the outcome of a collaborative process that includes the movement of large data, massive compute-intensive rendering tasks, and complex workflow executions.

2. Healthcare industry: In the healthcare industry community clouds are used to share information and knowledge on the global level with sensitive data in the private infrastructure.

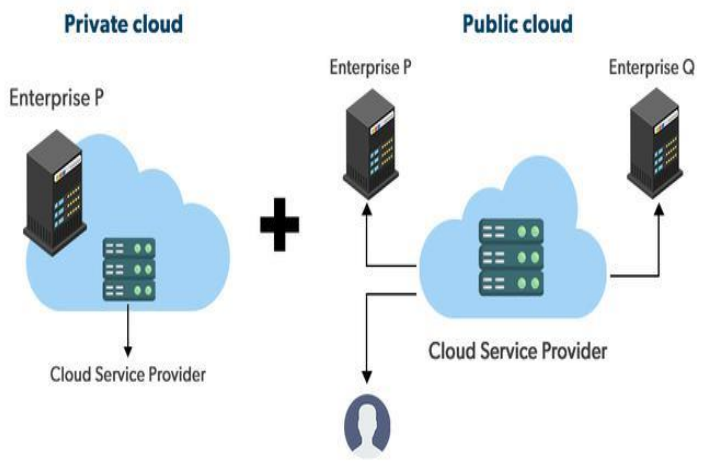
3. Energy and core industry: In these sectors, the community cloud is used to cluster a set of solution which collectively addresses the management, deployment, and orchestration of services and operations.

4. Scientific research: In this organization with common interests in science share a large distributed infrastructure for scientific computing.

(4) Hybrid cloud:

A hybrid cloud is a heterogeneous distributed system formed by combining facilities of the public cloud and private cloud. For this reason, they are also called **heterogeneous clouds.**

A major drawback of private deployments is the inability to scale on-demand and efficiently address peak loads. Here public clouds are needed. Hence, a hybrid cloud takes advantage of both public and private clouds.



Advantages of using a Hybrid cloud are:

1. **Cost:** Available at a cheap cost than other clouds because it is formed by a distributed system.
2. **Speed:** It is efficiently fast with lower cost, It reduces the latency of the data transfer process.
3. **Security:** Most important thing is security. A hybrid cloud is totally safe and secure because it works on the distributed system network.

Disadvantages of using a Hybrid cloud are:

1. It's possible that businesses lack the internal knowledge necessary to create such a hybrid environment. Managing security may also be more challenging. Different access levels and security considerations may apply in each environment.
2. Managing a hybrid cloud may be more difficult. With all of the alternatives and choices available today, not to mention the new PaaS components and technologies that will be released every day going forward, public cloud and migration to public cloud are already complicated enough. It could just feel like a step too far to include hybrid.

(c) Three Service Offering Models

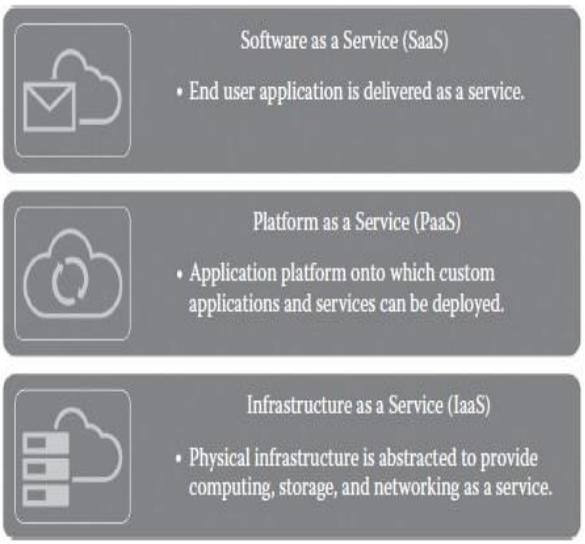


FIGURE 2.3
SPI—service offering model of the cloud.

Cloud computing is not a single piece of technology like a microchip or a cellphone. It's a system primarily comprised of three services:

(1) Infrastructure-as-a-service (IaaS), and
(2) Platform-as-a-service (PaaS).
(3) Software-as-a-service (SaaS)

(1) Infrastructure as a Service

- IaaS is also known as Hardware as a Service (HaaS). It is a computing infrastructure managed over the internet. The main advantage of using IaaS is that it helps users to avoid the cost and complexity of purchasing and managing the physical servers.

Characteristics of IaaS

There are the following characteristics of IaaS -

- Resources are available as a service
- Services are highly scalable
- Dynamic and flexible
- GUI and API-based access
- Automated administrative tasks

Companies providing IAAS are DigitalOcean, Linode, Amazon Web Services (AWS), Microsoft Azure, Google Compute Engine (GCE)

Example: AWS provides full control of virtualized hardware, memory, and storage. Servers, firewalls, and routers are provided, and a network topology can be configured by the tenant.,

(2) Platform as a service

- PaaS cloud computing platform is created for the programmer to develop, test, run, and manage the applications.

Characteristics of PaaS

There are the following characteristics of PaaS -

- Accessible to various users via the same development application.
- Integrates with web services and databases.
- Builds on virtualization technology, so resources can easily be scaled up or down as per the organization's need.
- Support multiple languages and frameworks.
- Provides an ability to "Auto-scale".

Companies offering PaaS are AWS Elastic Beanstalk, Windows Azure, Heroku, Force.com, Google App Engine, Apache Stratos, Magento Commerce Cloud, and OpenShift.

Example of PaaS is The World Wide Web (WWW) can be considered as the operating system for all our Internet-based applications. However, one has to understand that we will always need a local operating system in our computer to access webbased applications.

The basic meaning of the term *platform* is that it is the support on which applications run or give results to the users. For example, Microsoft Windows is a platform. But, a platform does not have to be an operating system. Java is a platform even though it is not an operating system.Through cloud computing, the web is becoming a platform. With trends (applications) such as Office 2.0, more and more applications that were originally available on desktop computers are now being converted into web–cloud applications. Word processors like Buzzword and office suites like Google Docs are now available in the cloud as their desktop counterparts.All these kinds of trends in providing applications via the cloud are turning cloud computing into a platform or to act as a platform.

(3)Software as a Service:

- SaaS is also known as "on-demand software". It is a software in which the applications are hosted by a cloud service provider. Users can access these applications with the help of internet connection and web browser.

Characteristics of SaaS

- There are the following characteristics of SaaS -
- Managed from a central location
- Hosted on a remote server
- Accessible over the internet
- Users are not responsible for hardware and software updates. Updates are applied automatically
- The services are purchased on the pay-as-per-use basis

Companies providing SaaS are BigCommerce, Google Apps, Salesforce, Dropbox, ZenDesk, Cisco WebEx, ZenDesk, Slack, and GoToMeeting.

Example of SaaS is The simplest thing that any computer does is allow us to store and retrieve information. We can store our family photographs, our favorite songs, or even save movies on it, which is also the most basic service offered by cloud computing. Let us look at the example of a popular application called *Flickr* to illustrate the meaning of this section.

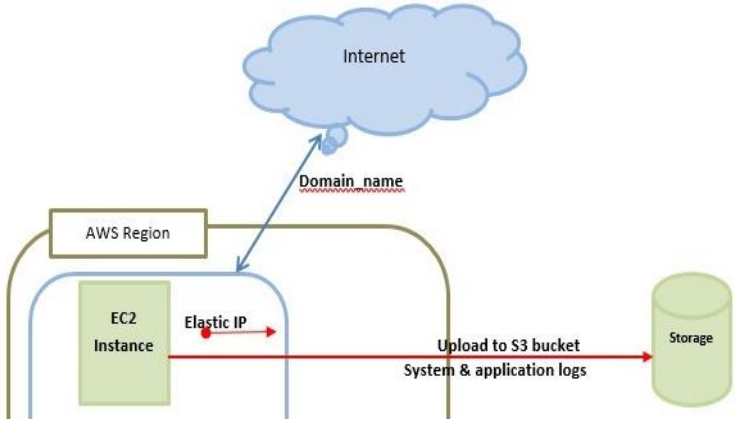
While Flickr started with an emphasis on sharing photos and images, it has emerged as a great place to store those images. In many ways, it is superior to storing the images on your computer:

1. First, Flickr allows us to easily access our images no matter where we are or what type of device we are using. While we might upload the photos of our vacation from our home computer, later, we can easily access them from our laptop at the office.
2. Second, Flickr lets us share the images. There is no need to burn them to a CD or save them on a flash drive. We can just send someone our Flickr address to share these photos or images.
3. Third, Flickr provides data security. By uploading the images to Flickr, we are providing ourselves with data security by creating a backup on the web. And, while it is always best to keep a local copy— either on a computer, a CD, or a flash drive—the truth is that we are far more likely to lose the images that we store locally than Flickr is of losing our images.

AMAZON ELASTIC COMPUTE CLOUD (EC2)

- EC2 which is a short form of Elastic Compute Cloud (ECC) it is a cloud computing service offered by the Cloud Service Provider AWS.
- Amazon EC2 (Elastic Compute Cloud) is a web service that acts as a scalable virtual servers in the cloud.
- It allows you to run applications, host websites, and process data with flexibility. You can choose from various instance types optimized for compute, memory, storage, or GPU needs.
- Instances are launched within a VPC (Virtual Private Cloud) and can use Elastic IPs for static public IP addresses.
- Security is managed using security groups, key pairs, and IAM roles.
- EC2 supports on-demand, reserved, and spot pricing, making it cost-effective. Persistent storage is provided via Elastic Block Store (EBS). Auto Scaling and Elastic Load Balancing ensure performance during traffic spikes.
- Monitoring tools like CloudWatch help track performance and usage. EC2's versatility makes it suitable for hosting, analytics, machine learning, and development tasks.
- Amazon EC2 can be used to launch as many or as few virtual servers as you need, configure security and networking, and manage storage. You can add capacity (scale up) to handle compute-heavy tasks,

such as monthly or yearly processes, or spikes in website traffic. When usage decreases, you can reduce capacity (scale down) again.



AWS EC2 Instance Types

Different Amazon EC2 instance types are designed for certain activities. Consider the unique requirements of your workloads and applications when choosing an instance type. This might include needs for computing, memory, or storage.The AWS EC2 Instance types are as follows:

- General Purpose Instances
- Compute Optimized Instances
- Memory-Optimized Instances
- Storage Optimized Instances
- Accelerated Computing Instances

1. General Purpose Instances

It provides the balanced resources for a wide range of workloads,It is suitable for web servers, development environments, and small databases. **Examples:** T3, M5 instances.

2. Compute Optimized Instances

It provides high-performance processors for compute-intensive applications.It will be Ideal for high-performance web servers, scientific modeling, and batch processing. **Examples:** C5, C6g instances.

3. Memory-Optimized Instances

High memory-to-CPU ratios for large data sets.Perfect for in-memory databases, real-time big data analytics, and high-performance computing (HPC). **Examples:** R5, X1e instances.

4. Storage Optimized Instances

It provides optimized resource of instance for high, sequential read and write access to large data sets.Best for data warehousing, Hadoop, and distributed file systems. **Examples:** I3, D2 instances.

5. Accelerated Computing Instances

It facilitates with providing hardware accelerators or co-processors for graphics processing and parallel computations. It is ideal for machine learning, gaming, and 3D rendering. **Examples:** P3, G4 instances.

Features of AWS EC2 (Elastic Compute Cloud)

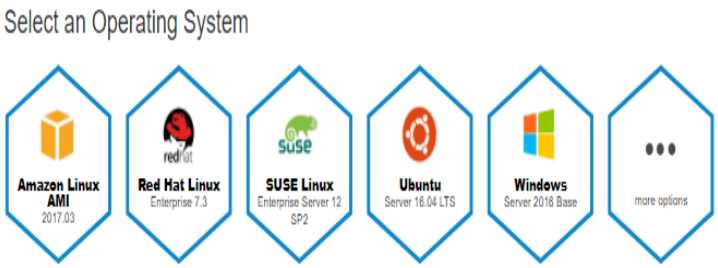
The following are the features of AWS EC2:

1. AWS EC2 Functionality

EC2 provides its users with a true virtual computing platform, where they can use various operations and even launch another EC2 instance from this virtually created environment. This will increase the security of the virtual devices. Not only creating but also EC2 allows us to customize our environment as per our requirements, at any point of time during the life span of the virtual machine. Amazon EC2 itself comes with a set of default AMI(Amazon Machine Image) options supporting various operating systems along with some pre-configured resources like RAM, ROM, storage, etc. Besides these AMI options, we can also create an AMI curated with a combination of default and user-defined configurations. And for future purposes, we can store this user-defined AMI, so that next time, the user won't have to re-configure a new AMI(Amazon Machine Image) from scratch. Rather than this whole process, the user can simply use the older reference while creating a new EC2 machine.

2. AWS EC2 Operating Systems

Amazon EC2 includes a wide range of operating systems to choose from while selecting your AMI. Not only are these selected options, but users are also even given the privilege to upload their own operating systems and opt for that while selecting AMI during launching an EC2 instance. Currently, AWS has the following most preferred set of operating systems available on the EC2 console.



3. AWS EC2 Software

Amazon is single-handedly ruling the cloud computing market, because of the variety of options available on EC2 for its users. It allows its users to choose from various software present to run on their EC2 machines. This whole service is allocated to AWS Marketplace on the AWS platform. Numerous software like SAP, LAMP, Drupal, etc are available on AWS to use.

4. AWS EC2 Scalability and Reliability

EC2 provides us the facility to scale up or scale down as per the needs. All dynamic scenarios can be easily tackled by EC2 with the help of this feature. And because of the flexibility of volumes and snapshots, it is highly reliable for its users. Due to the scalable nature of the machine, many organizations like Flipkart, and Amazon rely on these days whenever humongous traffic occurs on their portals.

5. Pricing of AWS EC2 (Elastic Compute Cloud) Instance

The pricing of AWS EC2-instance is mainly going to depend upon the type of instance you are going to choose. The following are the pricing charges on some of the EC2-Instances.

1. On-Demand Instances: The On-Demand instance is like a pay-as-you-go model where you have to pay only for the time you are going to use if the instance is stopped then the billing for that instance will be stopped when it was in the running state then you are going to be charged. The billing will be done based on the time EC2-Instance is running.
2. Reserved Instances: Reversed Instance is like you are going to give the commitment to the AWS by buying the instance for one year or more than one year by the requirement to your organization. Because you are giving one year of Commitment to the AWS they will discount the price on that instance.
3. Spot Instances: You have to bid the instances and who will win the bid they are going to get the instance for use but you can't save the data which is used in this type of instance.

Amazon EC2 Step by step Procedure:

1. Open **"EC2"** in services
2. Launch an instance from EC2 dashboard
3. Provide name to the instance
4. Select the Operating system also known as AMI **" ubuntu"**
5. Click on create a key pair
6. create a key pair name
7. A file will be downloaded. Save it for connect with SSH or Putty gen
8. In network settings create security group enable all the three SSH,HTTP,HTTPS.
9. Configure storage to 15GB (storage is our wish)
10. Click on launch instance
11. Successful initiation of instance
12. Goback to dash board of instances where an instance is initialized and is running then Right click onto the instance and click on connect it
13. Connect to instance, click on connect
14. Ec2 instance connect
15. Type sudo apt update
16. Type sudo apt install nginx
17. Do u want to continue: types yes
18. Copy and paste the public IP address in browser it opens nginx server

If the nginx not opened

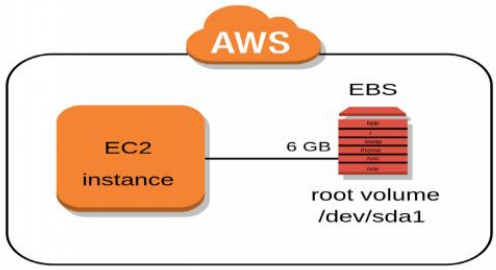
19. The site can't be reached so need to change the security settings
20. Go back to ec2 instance dashboard ,downside u have security tab
21. Click on security tab and select blue line security groups

22. The launch wizard will be as shown and edit the inbound rules
23. Add the rules -custom to "**Alltrafic"** and custom to **"anywhere ipv4"** and save rules
24. Saved changes and connect again then console will be opened.

RAPID ELASTICITY USING EBS IN CLOUD COMPUTING

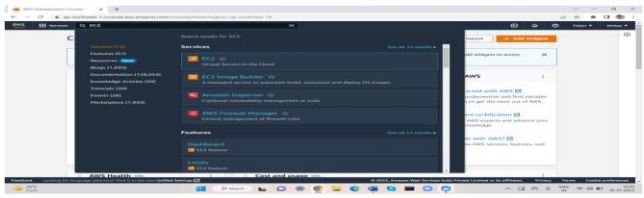
Elastic Block Storage (EBS): From the aforementioned list, EBS is a block type durable and persistent storage that can be attached to EC2 instances for additional storage. Unlike EC2 instance storage volumes which are suitable for holding temporary data EBS volumes are highly suitable for essential and long term data. EBS volumes are specific to availability zones and can only be attached to instances within the same availability zone.

EBS can be created from the EC2 dashboard in the console as well as in Step 4 of the EC2 launch. Just note that when creating EBS with EC2, the EBS volumes are created in the same availability zone as EC2, however when provisioned independently users can choose the AZ in which EBS is required.

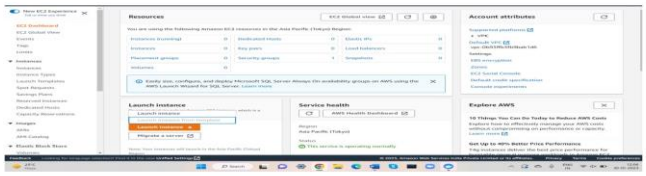


CREATION OF EC2 INSTANCE:

OPEN EC2 SERVICES



LAUNCH AN INSTANCE FROM EC2 DASHBOARD



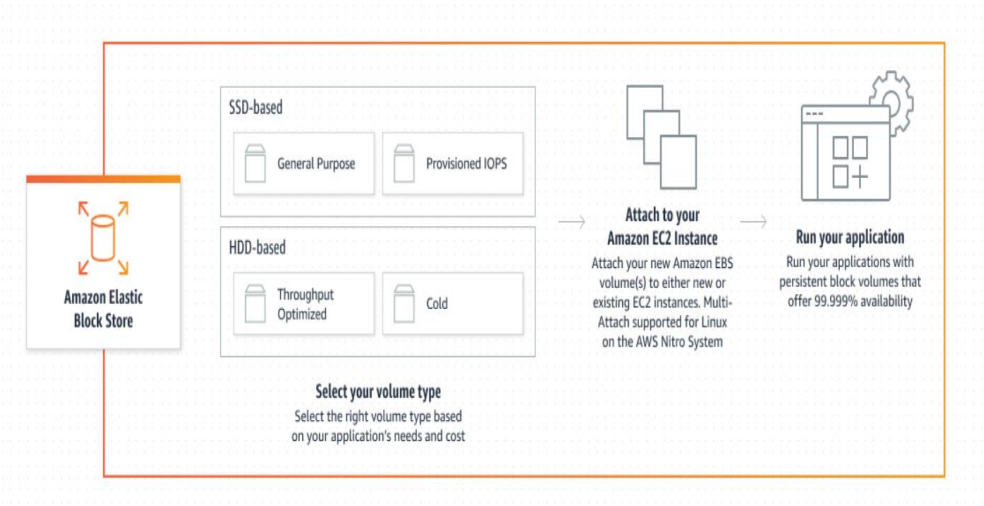


Fig: EBS Architecture

Features of EBS:

- **Scalability:** EBS volume sizes and features can be scaled as per the needs of the system. This can be done in two ways:
  - Take a snapshot of the volume and create a new volume using the Snapshot with new updated features.
  - Updating the existing EBS volume from the console.
- **Backup:** Users can create **snapshots** of EBS volumes that act as backups.
  - Snapshot can be created manually at any point in time or can be scheduled.
  - Snapshots are stored on AWS S3 and are charged according to the S3 storage charges.
  - Snapshots are incremental in nature.
  - New volumes across regions can be created from snapshots.
- **Encryption:** Encryption can be a basic requirement when it comes to storage. This can be due to the government of regulatory compliance. EBS offers an **AWS managed encryption feature.**
  - Users can enable encryption when creating EBS volumes by clicking on a checkbox.
  - Encryption Keys are managed by the Key Management Service (KMS) provided by AWS.
  - Encrypted volumes can only be attached to selected instance types.
  - Encryption uses the AES-256 algorithm.
  - Snapshots from encrypted volumes are encrypted and similarly, volumes created from snapshots are encrypted.
- **Charges:** Unlike AWS S3, where you are charged for the storage you consume, AWS charges users for the storage you hold. For example if you use 1 GB storage in a 5 GB volume, you'd still be charged for a 5 GB EBS volume.
  - EBS charges vary from region to region.EBS Volumes are independent of the EC2 instance they are attached to. The data in an EBS volume will remain unchanged even if the instance is rebooted or terminated.

Elasticity using EBS

- Elasticity is a 'rename' of scalability, a known non-functional requirement in IT architecture. Elasticity or Scalability is the ability to add or remove capacity, mostly processing, memory, or both, from an IT environment.
- Ability to dynamically scale the services provided directly to customers' need for space and other services. It is one of the five fundamental aspects of cloud computing.
- **Example :** Imagine a restaurant in an excellent location. It can accommodate up to 30 customers, including outdoor seating. Customers come and go throughout the day. Therefore restaurants rarely exceed their seating capacity. The restaurant increases and decreases its seating capacity within the limits of its seating area.

Scalability done in two ways:

Horizontal Scalability: Adding or removing nodes, servers, or instances to or from a pool, such as a cluster or a farm. Most implementations of scalability are implemented using the horizontal method, as it is the easiest to implement, especially in the current web-based world we live in.

Example : Adding Volumes of EBS to EC2 instance.

Vertical Scalability: Adding or removing resources to an existing node, server, or instance to increase the capacity of a node, server, or instance.Vertical Scaling is less dynamic because this requires reboots of systems, sometimes adding physical components to servers.Example : In Ec2 we are changing the t2.micro to t2.medium or t2.large.

Instance Type	vCPU	Memory (GiB)
t2.micro	1	1
t2.small	1	2
t2.medium	2	4

Let's look at some examples where we can use it.

Cloud Rapid Elasticity Example 1

Let us tell you that 10 servers are needed for a three-month project. The company can provide cloud services within minutes, pay a small monthly

We can compare this to before cloud computing became available. Let's say a customer comes to us with the same opportunity, and we have to move to fulfil the opportunity. We have to buy 10 more servers as a huge capital cost.

When the project is complete at the end of three months, we'll have servers left when we don't need them anymore. It's not economical, which could mean we have to forgo the opportunity.

Because cloud services are much more cost-efficient, we are more likely to take this opportunity, giving us an advantage over our competitors.

Cloud Rapid Elasticity Example 2

Let's say we are an eCommerce store. We're probably going to get more seasonal demand around Christmas time. We can automatically spin up new servers using cloud computing as demand grows.

New buyers will register new accounts. This will put a lot of load on your server during the campaign's duration compared to most times of the year.

Existing customers will also revisit abandoned trains from old wish lists or try to redeem accumulated points.It works to monitor the load on the CPU, memory, bandwidth of the server, etc. When it reaches a certain threshold, we can automatically add new servers to the pool to help meet demand. When demand drops again, we may have another lower limit below which we automatically shut down the server. We can use it to automatically move our resources in and out to meet current demand.

Cloud Rapid Elasticity Example 3

Streaming services, Netflix is probably the best example to use here. When the streaming service released all 13 episodes of House of Cards' second season, viewership jumped to 16% of Netflix's subscribers, compared to just 2% for the first season's premiere weekend.

Those subscribers streamed one of those episodes within seven to ten hours that Friday. Now, Netflix has over 50 million subscribers (February 2014). So a 16% jump in viewership means that over 8 million subscribers streamed a portion of the show in a single day within a workday.

Netflix engineers have repeatedly stated that they take advantage of the Elastic Cloud services by AWS to serve multiple such server requests within a short period and with zero downtime.

<u>EBS Volume types:</u>

(1)Solid State Disk Backed Volume

(a) General Purpose SSD (gp2)

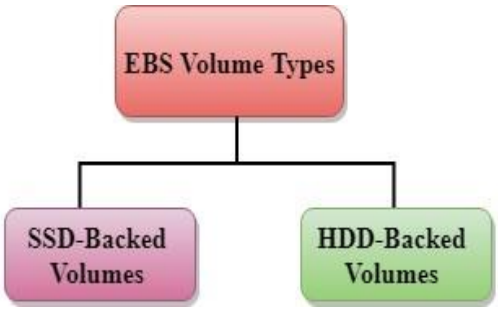
(b)Provisioned IOPS SSD (io1)

(2)Hard Disk Drive Backed Volume

(a) Throughput optimized HDD (st1)

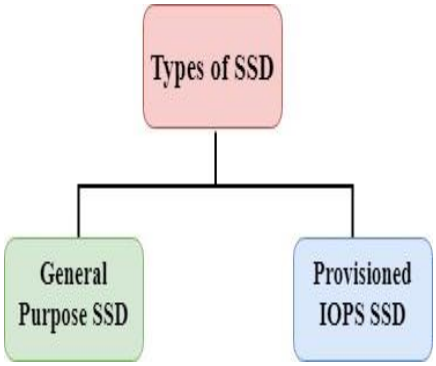
(b)Cold HDD

(cMagnetic standard(presently not available)



(1) SSD:

- SSD stands for solid-state Drives.
- In June 2014, SSD storage was introduced.
- It is a general purpose storage.
- It supports up to 4000 IOPS which is quite very high.
- SSD storage is very high performing, but it is quite expensive as compared to HDD (Hard Disk Drive) storage.
- SSD volume types are optimized for transactional workloads such as frequent read/write operations with small I/O size, where the performance attribute is IOPS.



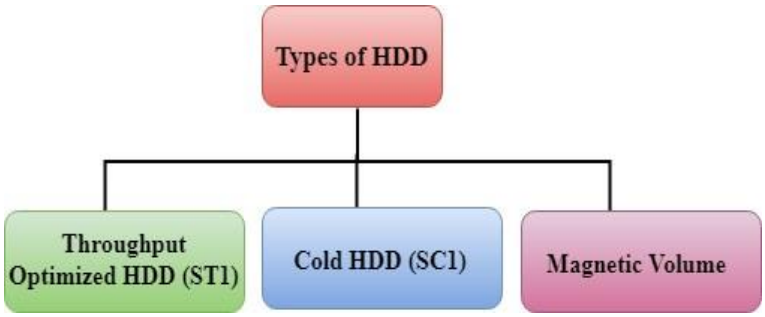
(a) General Purpose SSD (gp2)

- GP2 is the default EBS Volume type for the amazon EC2 instance.
- GP2 Volume are backed by SSDs.
- General purpose ,balances both price and performance .
- Ratio of 3 IOPS/GB with upto 10,000 IOPS • Both Volume having low latency
- Volume Size -4GB -16TB.
- Price - $0.10 gb/month.

(b) Provisioned IOPS SSD (io1)

- These volumes are for the IOPS intensive and throughput intensive workloads that require extremely low latency or for mission critical applications .
- Designed for I/O intensive application such as large relational or No SQL Databases.
- Use if you need more than 10,000 IOPS.
- Can provision upto 32,000 Iops per Volume (64,000 IOPS).
- Volume size 4gb -16Tb.
- Price - $0.125/gb/month.

(2) HDD:



(a) Throughput optimized HDD (st1)

- St1 is backed by hard disk drivers and is ideal for frequently accessed,
- Throughput intensive workloads with large datasets.
- It volume deliver performance in term of throughput , measured in MB/S.
- Big data, Data warehouse, log processing.
- It cannot be a Boot Volume can provisioned upto 500 IOPS per volume.
- Volume size – 500gb -16Tb.
- Prize - $0.045gb/month

(b) Cold HDD (SC1)

- SC1 is also backed by HDD and provides the lowest cost per gb of all EBS volume types
- Lowest cost storage for infrequent access workloads.
- Used in file servers.
- Cannot be a Boot Volume.
- Can provisioned upto 250 IOPS per volume.
- Volume size 500GB -16Tb.
- Price : $0- 025 /Gb/month.

(c)Magnetic Standard

- Lowest cost per Gb of all EBS volume type is Bootable.
- Magnetic volumes are Ideal for workloads when data is accessed
- infrequently ,and applications where the lowest storage cost is important .
- Price -$ 0.05 per gb/month
- Volume size - 1gb -1 TB
- Max IOPS/volume –40-200.

Step by step procedure for creating a volume using EBS in same zone:

- Create an empty EBS volume and attach it to a running instance at same availability zone of Ec2 instance.
- Create an EBS volume from a snapshot and attach it to a running instance in availability zone or from other zone.

(1) To create an EBS volume using the console in same availability zone

1. Open the Amazon EC2 console and Launch Ec2 Instance .
2. In the Left navigation pane, choose Elastic Block store..select **Volumes**.
3. Choose **Create volume**.
4. For **Volume type**, choose the type of volume to create select default ssd
5. For **Size**, enter the size of the volume, in GiB example 15GB.
6. For **Availability Zone**, choose the Availability Zone in which to create the volume. A volume can be attached only to an instance that is in the same Availability Zone.
7. After creating the volume ,attach it to running instance.

Note : The volume is ready for use when the **Volume state** is **available**.

8. Again In the left navigation pane, choose **Volumes**.
9. Select the volume which was created with 15GB ,enable it and choose **Actions , Attach volume**.

Note : You can attach only volumes that are in the Available state.

10. For **Instance**, enter the ID of the instance or select the instance from the list of options.

Note :.The volume must be attached to an instance in the same Availability Zone.

11. For **Device name**, enter a supported device name for the volume. This device name is used by Amazon EC2..
12. Choose **Attach volume**
13. Connect to the instance and mount the volume.
14. After that go t the instance dash board and click on storage then you will be able to see the attached EBS volume to the instance.

EBS snapshot:

- EBS snapshots are point -in-time images/copies of your EBS volume.
- Any data written to the volume after the snapshot process s initiated ,will not be included in the resulting snapshot (but will be included in future , incremental update).
- Per AWS account , up to 5000 EBS volume can be created.
- Per AWS account ,up to 10,000 EBS snapshot can be created.
- EBS snapshots are stored on S3 ,however you cannot access them directly you can only access them through EC2 APIs.
- While EBS volume are AZ specific , snapshot are Region specific.
- Any AZ in Region can use snapshot to create EBS volume.
- To migrate an EBS from one AZ to another Create a snapshot (region specific) and create an EBS volume from the snapshot in the intended AZ.
- You can create a snapshot to an EBS volume of the same or larger size than the original volume size from which the original volume size ,from which the snapshot was initially created.
- You can take a snapshot of a non-root EBS volume while the volume is in use on a Running EC2 instance.
- This means you can still access it while the snapshot is being processed.
- However the snapshot will only include data that is already written to your volume.
- The snapshot is created immediately but it may stay in pending status until the full snapshot is completed .This may takes, few hours to complete specially for the first time snapshot volume.
- During the period ,when the snapshot status is pending ,you can still success the volume (non-root), but I/O might be slower because of the snapshot activity.

- While in pending state ,on in-progress snapshot will not include data from ongoing reads and write to the volume.
- To take complete snapshot of your Non-root EBS volume.stop or unmount the volume.

To create a snapshot for a Root EBS volume ,you must stop the instance first then take the snapshot.

To create a snapshot using the console

Note: create 2 EC2 instances in 2 zones and create snapshot at one region and copy it to other destination.

(1)Creating instance in Ohio:

- Go to the EC2 dashboard and Create a new instance by launching it.
- The Zone is at Ohio region.
- Click on launch Instance to create instance
- Name the instance as cse1
- The OS must be Ubuntu
- Now create a new key pair
- Provide name to the key pair something as key1
- Click on create key pair and key pair is successfully created
- Click on Launch Instance
- Instance is successfully Launched

(2) Another instance in Virginia :

- Now in another tab change the zone to Virginia
- The Zone is successfully changed to Virginia
- Click on launch Instance to create instance
- Name the instance as cse2
- The OS must be Ubuntu
- Now click on create new key pair to create a new keypair
- Create a new key pair
- Provide name to the key pair something as key2
- Click on create key pair and key pair is successfully created
- Configure the storage from 8 to 10 • Click on Launch Instance
- Instance is successfully Launched

(3) Creation of snapshot

- Go back to the instance 1 dashboard ,in the left pane at EBS select snapshot.
- Now in actions click on create a snapshot
- Now create a snapshot
- Add the description something as snapshot1 and create snapshot

- Snapshot is created successfully
- Now go to the snapshots
- The snapshot created will be shown. Now select the snapshot and go to actions
- Select copy snapshot option
- Now in copy snapshot add destination region as us-east1
- Snapshot is created successfully

(4)Accessing from second zone:

- Now go to the second instance created. In that check for snapshots
- We will find our snapshot that we created and status will be completed

Advantages and Disadvantages of Rapid Elasticity

Advantages

Rapid elasticity in cloud computing provides an array of advantages to businesses hoping to scale their resources.

- **High availability and reliability.** With rapid elasticity, you can enjoy a remarkably consistent, predictable experience. Cloud providers take care of scaling behind the scenes, keeping the system running smoothly and fast.
- **Growth-supporting.** You can more easily adopt a growth-oriented mindset with rapid elasticity. With elastic cloud computing, your IT infrastructure becomes more agile and nimble, as well as more prepared to acquire new users and customers.
- **Automation capability.** Rapid elasticity in cloud computing uses increased automation in your IT environment, which has many benefits. For example, you can free up your IT staff to focus on core business functionality rather than scalability.
- **Cost-effective.** Cloud providers offer resources on a pay-per-use basis, so you only pay for what you actually use. Adding new infrastructure components to prepare for growth becomes convenient with a pay-as-you-expand model.

Disadvantages

Though rapid elasticity in cloud computing provides a multitude of benefits, it also introduces a few complexities you should keep in mind.

- **Learning curve.** Rapid elasticity takes some time and effort to fully comprehend and therefore benefit from. Your staff may need to familiarize themselves with new programming languages, cloud platforms, automation tools, etc.
- **Security.** Since elastic systems only run for a short period, you must rethink how you handle user authentication, incident response, root cause analysis, and forensics when you are dealing with

security. Luckily, experts like Synopsys provide accessible and reliable cloud security solutions to simplify this process.

- **Cloud lock-in.** Rapid elasticity is a big selling point for public cloud providers, but vendors can lock you into their service. Do your research before settling on a public cloud provider to ensure you fully understand its offerings and your contract.

AMAZON ELASTIC FILE SYSTEM (AMAZON EFS)

- AWS Elastic File System (EFS) is a fully managed, scalable, and elastic file storage service provided by Amazon Web Services.
- It is designed to provide file storage that can be shared across multiple Amazon EC2 instances
- EFS is particularly well-suited for applications that require access to a shared file system or need to process large amounts of data in parallel.
- EFS can be mounted to different AWS services,You'll always pay for the storage you actually use, rather than provisioning storage in advance that's potentially wasted.
- EFS can be created using the EC2-Instance where it will be created in a specific region and distributed across multiple availability zones for the purpose of high availability and durability.
- Amazon EFS is designed to be highly available and durable for thousands of EC2 instances that are connected to the service. Amazon EFS stores each file system object in multiple availability zones (AZs); an IT professional can access each file system from different AZs in the region it is located. The service also supports periodic backups from on-premises storage services to EFS for disaster recovery.
- The Network File System v4.1 protocol mounts an EFS system on an EC2 instance or an on-premises server to give the service access to data and to enable it to read and write to the file system.
- Once the EFS is created you need to set up mount targets which will provide the connectivity to your EFS file system. Following are some of the resources which you can mount on EFS.
- Amazon EC2.
- Amazon ECS.
- Amazon EKS.
- AWS Fargate.
- AWS lambda and some other servers.

Amazon Elastic File System (Amazon EFS)

Fig ; Amazon EFS Architecture

Use Cases Of EFS:

1. Secured file sharing: You can share your files in every secured manner and in a faster and easier way and also ensures consistency across the system.
2. Web Hosting: Well suited for web servers where multiple web servers can access the file system and can store the data EFS also scales whenever the data incoming is increased.
3. Modernize application development: You can share the data from the AWS resources like ECS, EKS, and any serverless web applications in an efficient manner and without more management required.
4. Machine Learning and AI Workloads: EFS is well suited for large data AI applications where multiple instances and containers will access the same data improving collaboration and reducing data duplication.

Step by Step Procedure of EFS

Note: Select OS Linux while doing EFS , use same security group and key pair for both instances and select the subnet id for second instance other than first instance while creating security group

(1) Creating first instance and note down the availability zone and security group

- Create the first instance by launching the Instance
- Name the first instance to be launched as efs1
- Select The operating System as AWS Linux
- Create a Key pair and name it as nfs
- Create the key pair.
- Configure the network settings
- Provide all the permissions as shown by checking the boxes.

- Configure the storage from 8GB to 10GB
- Now Launch the instance.
- Instance 1 named efs1 is successfully launched.

(2)Creating Second instance and adding same security group, keypair and different availability zone

- Go back to the Dashboard of EC2 instance to create another instance.
- Check the availability zone and Security group in another tab so that for the second instance the availabilty zone should be different and security group to be the same.
- Repeat the same process and create another instance with name as efs2 same as the previous instance and use same key and security group for instance 2.
- Checking the Security group in another tab so that to apply it same to the instance 2 that we are creating.
- Changing the security group as wizard2 as it was in instance1
- Now edit the same for the second instance2
- Select the existing security group for instance2
- Launch the instance2
- Instance 2 is also successfully launched
- The availability zones are different.

(3) Adding security group New Rule with NFS

- The Security groups are same, if we add both instances can be reflected.
- Select the first instance efs1
- Click on the security tab in dashboard of instance of efs1 click on the security groups
- Now select the inbound rules and edit the inbound rules,in the edit inbound rules click on add rule
- Add the rule , Select NFS and anywhere in IPV4.
- Save the changes and the saved changes will be successful.

(4) Creating EFS Service

- Go back to the dashboard and search for EFS, Now click on create file system
- Provide name to the file system something as efsdemo ,Let VPC be default and Storage class as Standard and click on customize
- Click on Next and Removed all the previous provided Security Groups Network access.
- Apply the security group name same as the EC2 instances security group
- Click on next, Now click on Create.
- EFS created successfully.

(5)Mounting the EFS with instances from console

- Go back to instances
- Now go to efs1 i.e, instance 1 and right click the instance and connect to the instance.
- Connect to the instance, by Click on connect to establish connection
- Connection is being established
- After connection is established, Same step must be repeated for the second instance efs2 and connection must be established.
- After connection is established, Type the below all commands in two instance consoles

  1) sudo su

  2) mkdir efs

  3) yum install-y amazon-efs-utils

- Go back to the amazon aws console , in the services go back to efs service, right click on created efs i.e. efsdemo
- Click on attach to mount efs,We are ,mounting via DNS ,Copying the command and paste in two consoles.

(6) Creating EFS directory and files

- Type the commands in two consoles

  - ls

  - cd efs

- Now create a file in any **one of the ec2 instance** such that **it must reflect in another instance** even.

  For example create file in instance1 must reflect in instance 2

- Type the command in one console

  - touch file1 file2

  - ls in any one of the instance.

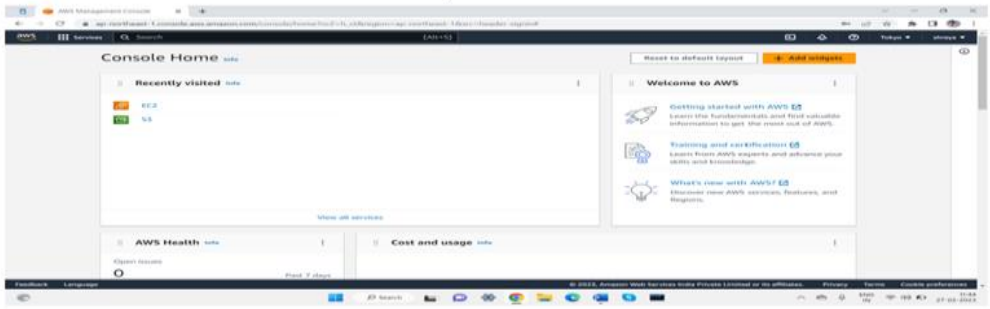
- Touch file1 file2 to create files and ls to list the created files.
- In another instance(instance where touch command not used) type ls. It shows the created files l1 and l2
- In the other instance ,to remove the file type the command

  - rm file1

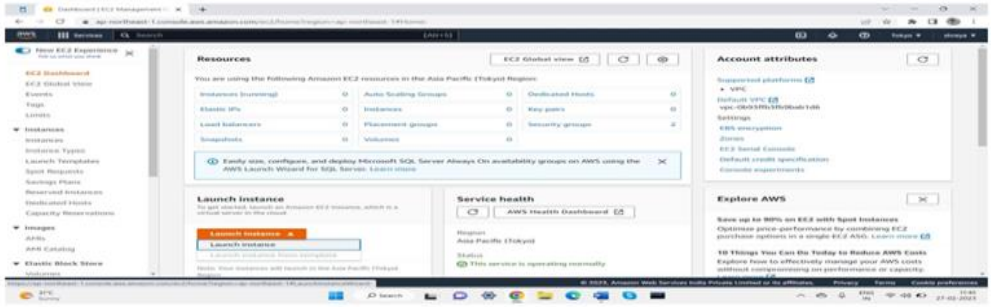
  - ls

Check the same ls command in the instance where we have created file1 and file2 after removal of file1. It shows only file2 In this process Efs can be shared among ec2 instances with in the regions.

In the console home of the AWS dashboard open EC2 instance.



Create the first instance by launching the Instance



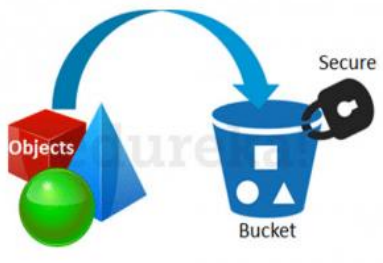
AMAZON S3

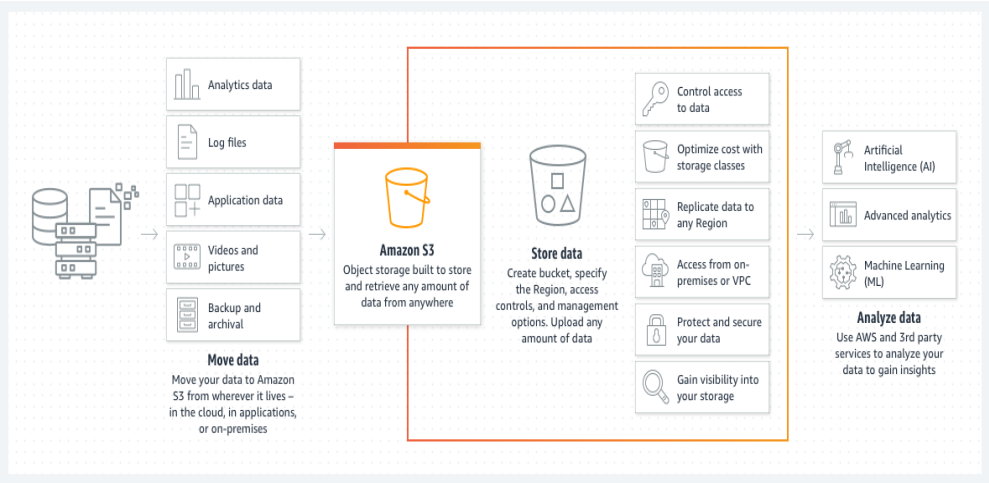
- Amazon Simple Storage Service (S3) is a storage for the internet. It is designed for large-capacity, low-cost storage provision across multiple geographical regions.
- Amazon S3 provides object storage with its own unique identifier or key, for access through web requests from any location .
- Unlike EBS or EFS, S3 is not limited to EC2.
- Files stored and protected in S3 bucket can be accessed by customers directly or programmatically of all sized industries.

How Amazon S3 works:

- Data in S3 is organized in the form of buckets.
- Amazon S3 is an object storage service that stores data as objects within buckets. An *object* is a file and any metadata that describes the file. A *bucket* is a container for objects.

- To store your data in Amazon S3, you first create a bucket and specify a bucket name and AWS Region. Then, you upload your data to that bucket as objects in Amazon S3. Each object has a *key* (or *key name*), which is the unique identifier for the object within the bucket.
- S3 provides features that you can configure to support your specific use case. For example, you can use S3 Versioning to keep multiple versions of an object in the same bucket, which allows you to restore objects that are accidentally deleted or overwritten.
- Buckets and the objects in them are private and can be accessed only if you explicitly grant access permissions. You can use bucket policies, AWS Identity and Access Management (IAM) policies, access control lists (ACLs), and S3 Access Points to manage access.





S3 Architecture

S3 Buckets –Naming Rules

- S3 Bucket names (keys) are globally Unique across all AWS Regions.
- Bucket Names cannot be change after they are created.
- If a bucket is deleted ,its name becomes available again to your or other account to use.
- Bucket names must be at least 3 and no more than 63 characters long.
- Bucket names are part of the URL used to access a bucket.
- Bucket name must be a services of one or more labels (xyz bucket) .
- Bucket names can contain lowercase , numbers and cannot use uppercase letters.

- Bucket name should not be an IP address(10 to 2020)
- Each label must start and end with a lowercase letter or a number
- By default buckets and its objects are private by default, only owner can access the bucket.
- The name is of two parts :-

Bucket region's endpoint /bucket name

Ex: S3 bucket named mybucket in Europe west Region

https://S3-eu-west1.amazons.com/mybucket

S3 Buckets-Sub resources:

Sub-resources for S3 bucket includes:-

- Lifecycle:- To decide on objects lifecycle management
- Website:- To hold configuration related to state website hosted in S3 buckets.
- versioning :- Keep object versions as it changes (Gets updated)
- Cross-region replication:- Automate , fast and reliable asynchronous replication of data
- across region
- Access Control List:- Buckets Policies.

S3 Objects

Any object size stored in an S3 bucket can be) bytes to 5 TB.

- Each object is stored and retrieved by a unique key(ID or name).
- An object in AWS S3 is Uniquely identified and addressed through

  -Service endpoint

  -Bucket name

  -Optionally Object Version
- Object stored in a S3-bucket in a Region will never leave that region unless you specifically move them to another region or CRR.
- A bucket owner can grant cross-account permission to another AWS account to upload objects.
- We can grant S3 bucket /object permission to :-

  -Individual users

  -AWS Account

  -Make the Resource public

  -or to all authenticate user

S3 Bucket versioning

- Bucket versioning is a S3 bucket Sub-resource used to protect against accidental object /data deleted or overwrites.
- Versioning can also be used for data Retention and archive.
- Once you enable versioning on a Buckets ,it cannot be disabled ,however it can be suspended.

- When enabled ,bucket versioning will protect existing and new objects, and maintains their versions as they are updated.
- Updating objects refers to PUT,POST,COPY,DELETE actions on objects.
- When versioning is enabled and you by to delete an object , a delete marker is placed on the object.
- We can still view the object and the delete marker.
- It you Reconsider deleting the objects ,we can delete the "Delete Marker" and the object will be available again.
- We will be changed for all S3 storage cost for all object versions stored.
- We can use versioning with S3 lifecycle policies to delete older versions, or you move them to a cheaper S3 storage (or Glacier).
- Bucket versioning state
  -Enabled
  -Suspended
- Versioning applies to all objects in a bucket and not partially applied.
- Object Existing before enabling versioning will have a version ID.
- If you have a bucket that is already versioned ,then you suspend versioning, existing objects and their versions remain as it is.
- However they will not be updated/versioning further with future updates while the bucket versioning is suspended.
- New object (uploaded after suspension they will have a version ID "null"
- If the same key (name) is used to stone another object ,it will override the existing one.
- An object deletion in a suspended versioning buckets will only delete the objects with ID "null".

S3 Cross-Region Replication

- Replication enables automatic, asynchronous copying of objects across Amazon S3 buckets.
- Buckets that are configured for object replication can be owned by the same AWS account or by different accounts.
- You can replicate objects to a single destination bucket or to multiple destination buckets.
- The destination buckets can be in different AWS Regions or within the same Region as the source bucket.
- To automatically replicate new objects as they are written to the bucket, use live replication, such as Cross-Region Replication (CRR).
- To enable CRR, you add a replication configuration to your source bucket To enable CRR, you add a replication configuration to your source bucket.
  -The destination bucket or buckets where you want Amazon S3 to replicate objects
  -An AWS Identity and Access Management (IAM) role that Amazon S3 can assume to replicate objects on your behalf

Use Cases :

- Compliance – store data hundreds of miles a part
- Lower latency- Distribute data to regional customers
- Security – create remote replicas managed by separate AWS accounts
- Only replicates new PUTs once S3 is configured ,all new updates into a source buckets will be replicated.
- Versioning is required.

To upload objects in Amazon S3:

First u need to create Bucket with the following points

- Create Buckets
- Upload Objects
- S3 Versioning
- Version ID
- Bucket Policy
- Access Control List (ACL's)

Step by step procedure to create S3 bucket:

1.Click on S3 services.

2. Provide bucket name and enable the acl's enabled.

3. Unblock all the public access settings for the bucket

4.Check box the acknowledgement, disable bucket versioning.

5.Provide default encryption as amazon s3-managed keys and bucket key as enable and click on create bucket.

6.Bucket is successfully created.

7. After creation of bucket ,click on the bucket and go to permission tab, enable the ACL's permission to list and read.

8. Now right click on the bucket created, click on upload to upload A file here I uploaded cse.Jpg

9.Click on upload and the file is uploaded successfully.

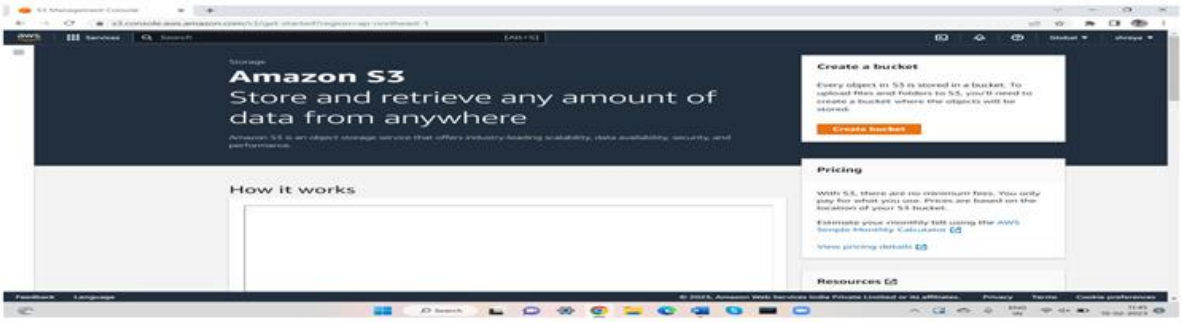
10. After upload of file once again go to permission tab of object check whether ACL's permission to list and read is enabled or not.

11.Now go back to the file been uploaded into the bucket

12.Right click onto the file and go to properties, Copy the object url and paste in browser,will be able to see the object uploaded publicly.

OPEN S3

CLICK ON S3 SERVICES



AWS S3 Bucket Benefits

- Users get 99.99% durability.
- New users get 5GB of Amazon S3 standard storage.
- S3 provides Encryption to the data that you store. In two ways, Client-Side Encryption and Server-Side Encryption
- Multiple copies are maintained to enable the regeneration of data in case of data corruption.
- S3 is Highly Scalable since it automatically scales your storage according to your requirement. only pay for the storage you use.

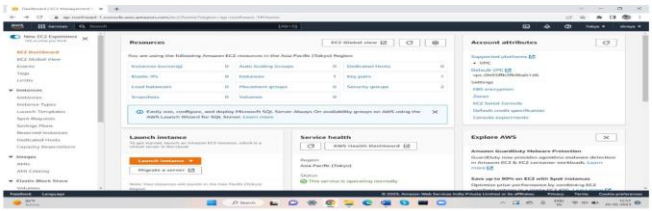
Amazon S3 Use Cases

- **Data lake and big data analytics:** S3 can create a data lake to hold raw data in its native format, then use machine learning tools, query-in-place, and analytics to draw insights. S3 works with AWS Lake Formation to create data lakes, then define governance, security, and auditing policies. Together, they can be scaled to meet your growing data stores, and you'll never have to make an investment upfront.

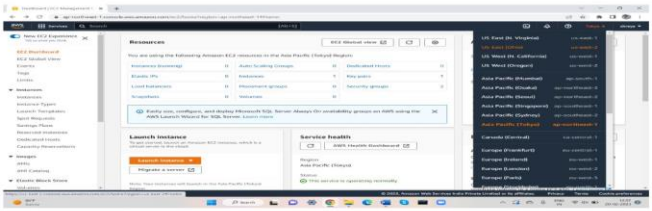
- **Backup and restoration:** Secure, robust backup and restoration solutions are easy to build when you combine S3 with other AWS offerings, including EBS, EFS, or S3 Glacier. These offerings enhance your on-premises capabilities, while other offerings can help you meet compliance, recovery time, and recovery point objectives.

- **Reliable disaster recovery:** S3 storage, S3 Cross-Region Replication and additional AWS networking, computing, and database services make it easy to protect critical applications, data, and IT systems. It offers nimble recovery from outages, no matter if they are caused by system failures, natural disasters, or human error.
- **Methodical archiving:** S3 works seamlessly with other AWS offerings to provide methodical archiving capabilities. S3 Glacier and S3 Glacier Deep Archive enable you to archive data and retire physical infrastructure. There are three S3 storage classes you can use to retain objects for extended periods of time at their lowest rates. S3 Lifecycle policies can be created to archive objects at any point within

Go to the EC2 dashboard and Create a new instance by launching it.



Changing of the Zone from Tokyo to Ohio



their lifecycle, or you can upload objects to archival storage classes directly. S3 Object Lock meets compliance regulations by applying retention dates objects to avoid their deletion. And unlike a tape library, S3 Glacier can restore any archived object within minutes.

Amazon S3 storage classes Types

Amazon S3 offers different storage classes with different levels of durability, availability, and performance requirements.

- Amazon S3 Standard is the default
- Amazon S3 Standard-Infrequent Access (Standard-IA) –Infrequent objects can be stored.
- Amazon S3 One Zone-Infrequent Access (One Zone-IA) - Infrequent objects can be stored in one zone
- Amazon S3 on Outposts -will give users 48TB or 96TB of S3 storage capacity, with up 100 buckets on each Outpost.
- Amazon S3 Glacier Deep Archive - ideal for those industries which store data for 5-10 years or longer like healthcare, finance, etc. It can also be used for backup and disaster recovery.

Note : Mostly Amazon s3 Standard is default all are using.

Advantages and Disadvantages of Rapid Elasticity

Advantages

Rapid elasticity in cloud computing provides an array of advantages to businesses hoping to scale their resources.

•**High availability and reliability.** With rapid elasticity, you can enjoy a remarkably consistent, predictable experience. Cloud providers take care of scaling behind the scenes, keeping the system running smoothly and fast.

•**Growth-supporting.** You can more easily adopt a growth-oriented mindset with rapid elasticity. With elastic cloud computing, your IT infrastructure becomes more agile and nimble, as well as more prepared to acquire new users and customers.

•**Automation capability.** Rapid elasticity in cloud computing uses increased automation in your IT environment, which has many benefits. For example, you can free up your IT staff to focus on core business functionality rather than scalability.

•**Cost-effective.** Cloud providers offer resources on a pay-per-use basis, so you only pay for what you actually use. Adding new infrastructure components to prepare for growth becomes convenient with a pay-as-youexpand model.

Disadvantages

Though rapid elasticity in cloud computing provides a multitude of benefits, it also introduces a few complexities you should keep in mind.

•**Learning curve.** Rapid elasticity takes some time and effort to fully comprehend and therefore benefit from. Your staff may need to familiarize themselves with new programming languages, cloud platforms, automation tools, etc.

•**Security.** Since elastic systems only run for a short period, you must rethink how you handle user authentication, incident response, root cause analysis, and forensics when you are dealing with security. Luckily, experts like Synopsys provide accessible and reliable cloud security solutions to simplify this process.

•**Cloud lock-in.** Rapid elasticity is a big selling point for public cloud providers, but vendors can lock you into their service. Do your research before settling on a public cloud provider to ensure you fully understand its offerings and your contract.

AMAZON LEX

Amazon Lex is an AWS service for building conversational interfaces for applications using voice and text. With Amazon Lex, the same conversational engine that powers Amazon Alexa is now available to any developer, enabling you to build sophisticated, natural language chatbots into your new and existing applications.

Amazon Lex provides the deep functionality and flexibility of natural language understanding (NLU) and automatic speech recognition (ASR) so you can build highly engaging user experiences with lifelike, conversational interactions, and create new categories of products.

Amazon Lex: How It Works

Amazon Lex enables you to build applications using a speech or text interface powered by the same technology that powers Amazon Alexa. Following are the typical steps you perform when working with Amazon Lex:

1. Create a bot and configure it with one or more intents that you want to support. Configure the bot so it understands the user's goal (intent), engages in conversation with the user to elicit information, and fulfills the user's intent.
2. Test the bot. You can use the test window client provided by the Amazon Lex console.
3. Publish a version and create an alias.
4. Deploy the bot. You can deploy the bot on platforms such as mobile applications or messaging platforms such as Facebook Messenger.

Before you get started, familiarize yourself with the following Amazon Lex core concepts and terminology:

- **Bot** – A bot performs automated tasks such as ordering a pizza, booking a hotel, ordering flowers, and so on. An Amazon Lex bot is powered by Automatic Speech Recognition (ASR) and Natural Language Understanding (NLU) capabilities. Each bot must have a unique name within your account.Amazon Lex bots can understand user input provided with text or speech and converse in natural language. You can create Lambda functions and add them as code hooks in your intent configuration to perform user data validation and fulfillment tasks.
- **Intent** – An intent represents an action that the user wants to perform. You create a bot to support one or more related intents. For example, you might create a bot that orders pizza and drinks. For each intent, you provide the following required information:
- **Intent name**– A descriptive name for the intent. For example, order pizza Intent names must be unique within your account.
- **Sample utterances** – How a user might convey the intent. For example, a user might say "Can I order a pizza please" or "I want to order a pizza".

How to fulfill the intent – How you want to fulfill the intent after the user provides the necessary information (for example, place order with a local pizza shop). We recommend that you create a Lambda function to fulfill the intent.

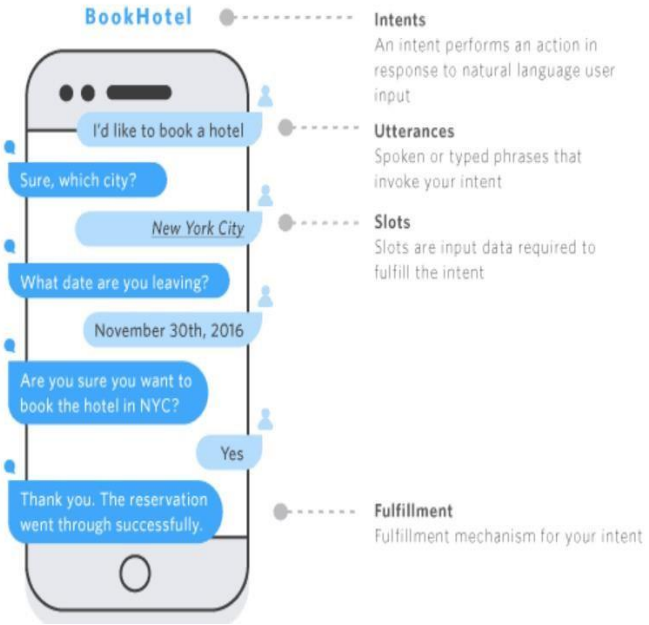
For example, the Order Pizza intent requires slots such as pizza size, crust type, and number of pizzas. In the intent configuration, you add these slots. For each slot, you provide slot type and a prompt for Amazon Lex to send to the client to elicit data from the user. A user can reply with a slot value that includes additional words, such as "large pizza please" or "let's stick with small." Amazon Lex can still understand the intended slot value.

Slot type – Each slot has a type. You can create your custom slot types or use built-in slot types. Each slot type must have a unique name within your account. For example, you might create and use the following slot types for the order pizza intend.

Size – With enumeration values small, Medium, and Large.

Crust – With enumeration values Thick and Thin.

Amazon Lex also provides built-in slot types For example, AMAZON .NUMBER. is a built-in slot type that you can use for the number of pizzas ordered For more information.



Intend :-An intend represents an action that the user wants to perform

For example ,you might create an intend that orders pizza,books ,check balance ,apply for a loan, payment issue, etc

Sample Utterances – How a user might convey the intent.

For example, a user might stay. ”Can I order a pizza” or “I want to order a pizza”.

Slot – A slot is an information that Amazon Lex needs to fulfil an intend. Each slot has type. You can create your custom slot types or use built-in types.

For example ,the OrderPizza intend intent requires slots such as pizza and pizza type.

Slot type – Each slot has a type . You can create your slot type or use built –in slot types.

For example you might create and use the following slot types for the OrderPizza intent:

Size – With enumeration values Small , Medium and Large.

Crust – With enumeration values Thick and Thin.

Hotel Booking Chatbot

Intend

BookHotel

- **Sample Utterances** Book a Hotel

I want a make hotel reservations

Book a 3 nights stay in Mumbai

Book a {nights} nights stay in {location}

- **Slots**

  Location

  Prompts : What city will you be staying in ?

- **CheckinDate**

  Prompts: What day do you want to check in ?

- **Nights**

  Prompts: How many nights will you be staying?

- **RoomType**

  Prompts: What type of room you will like queen, king, or deluxe?

- **Configuration prompt**

  Okay , I have you done {Nights} night stay in {Location} starting {checkinDate}. Shall I book the reservation?

- **Decline response**

  Okay, I have cancelled your reservation in progress.

<u>Step by step Procedure of Lex:</u>

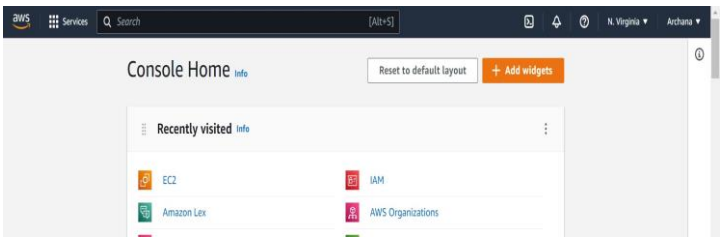
- Select Amazon Lex service from console
- Now here click on -> create bot
- Here we have to select the Create a blank bot
- Give the name to the Bot here I have given BookingHotel
- we have select the create a role with basic Amazon Lex permission
- we have to give the children online privacy protection Act as No
- Select the language as English US
- We have different options ,here we select the “This is only text based application”
- We give the name for Intend, name as BookHotel
- Here we add the **Sample Utterances**

- Book a Hotel
- I want a make hotel reservations
- Book a {nights} nights stay in {location} sample :Book a 3 nights stay in Mumbai

- Here we will give the prompts for utterances
- We can add more extra questions by adding in ADD Slot
- Here we are trying to add extra slot , where it is not predefined

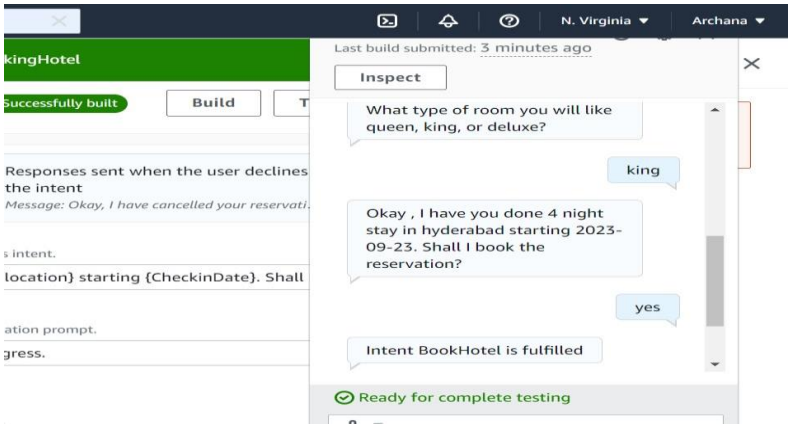
- We can create the our own slot by creating the slot types
- Go to intend ,slot types-□□we click on Add Slot types
- Here we give the slot type
- After giving the name for the slot here we go for restrict to slot values -> means it
- take those values which we have given to them
- Here we give the values for the slot type ,and click on create slot
- Now come to the intend, go to slots click on add slot
- Here we can see the our slot type which we have created ,add prompts also
- After adding all the required slots we can click on save intend
- Here will get the preview after saving the intend, n build the intend
- After that build it ,we go for test option ,right side panel will be created with chat
- Now we can give any one of the utterance here, I gave here Book Hotel, you see response also.
- In response to Book a hotel we get the replay from bot, in this way will get the responses
- At last we get intend BookHotel is fulfilled, in spite of this message we change it .
- Go to the intend ,□□confirmation □prompts □□confirmation prompts □□type your message
- Save the intend ,build it and test it again
- Now we can the confirmation message is displayed here

Select Amazon Lex service from console



Now here click on -> create bot

Now we can the confirmation message is displayed here



AWS LAMBDA

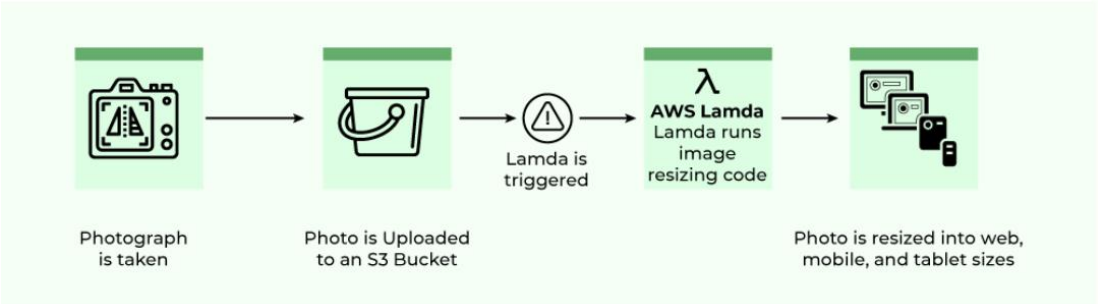
- Aws lambda is a compute service that let you run code without provisioning or managing servers virtually any type of application with **zero administration.**
- You pay only for the compute time consume no charge when your code is not running.
- All you need to do is supply your code in the form of one or more lambda functions to aws lambda in one of the languages that aws supports (currently Node js , Java , power shell , c#, ruby ,python and Go lang) and the services can the code on your behalf
- Typically the lifecycle for an aws lambda based application includes authoring code ,deploying code to aws lambda and then monitoring and troubleshooting.

AWS lambda manages the following:

- Provisioning and compute fleet that offers a balance of memory cpu n/w and other services.
- Server and OS maintenance.
- High availability and automatic scaling.
- Monitoring fleet health.
- Applying security patches.
- Deploying your code and trigger your code by using lambda function.
- Aws lambda runs your code on a high-availability compute infrastructure.
- Aws lambda executes your code only when needed and scales automatically from a few requests per day to thousand per second.
- You pay only for the compute time consume no charge when your code is not running.
- All you need to do is supply your code in the form of one or more lambda functions to aws lambda in one of the languages that aws supports (currently Node js , Java , power shell , c#, ruby ,python and Go lang) and the services can the code on your behalf
- Typically the lifecycle for an aws lambda based application includes authoring code ,deploying code to aws lambda and then monitoring and troubleshooting.
- This is in exchange for flexibility which means you cannot log into compute instance or customize the operating the operating system or lang runtime.
- If you do want to manage your own compute you can use ec2 or elastic bean stalk.

How lambda works

- First you upload your code to lambda in one/more lambda function.
- Aws lambda will then execute the code you behalf.
- After the code is invoked lambda automatically take care of provisioning and managing the required servers.



Maximum execution timeout of Lambda:

- You pay for the AWS resources that are used to run your lambda function
- To prevent your lambda function from running indefinitely , you specify a timeout
- When the specified timeout is reached , aws lambda terminates your lambda function
- Default is 3 sec and maximum is 900 sec.(15 minutes)

Different ways of using lambda function

- Synchronous invoke (push)
- Asynchronous invoke (event)
- Poll-based invoke (pull based)

(a) Synchronous invoke

one the most straight forward way to make your lambda ,In this model ,your function executes immediately when you perform the lambda invoke API call.

Invocation flag specify a value of 'Request Response' to process the event and return a repones.

- Here is a list of services that involve synchronous lambda function synchronously

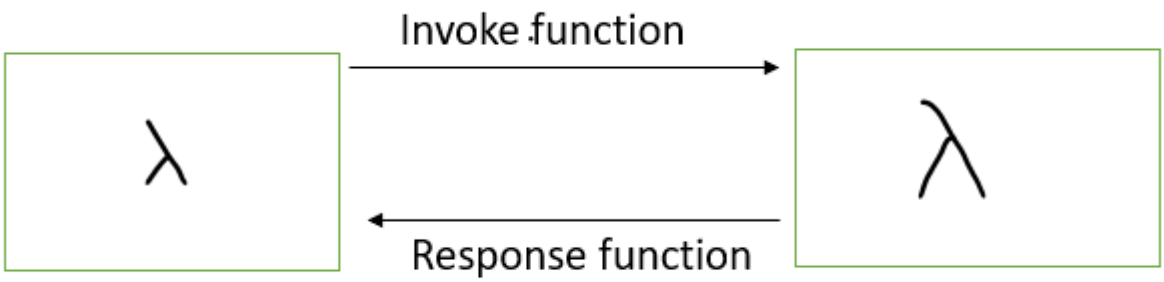
Elastic load balancer

Amazon Coginto

API gateway

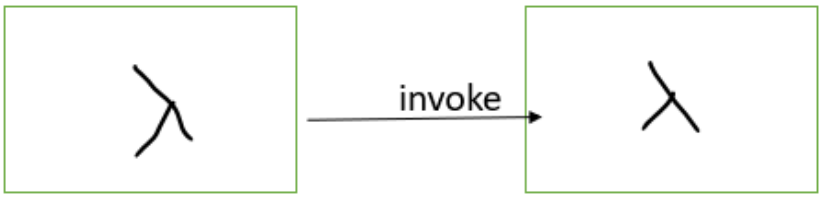
Amazon Lex

Kinesis Data Firehose



(b) Asynchronous Invocation

- For asynchronous invocation ,lambda places the event in a Queue and returns a success response without additional information.
- Lambda queues the event for processing and returns a response immediately.
- You can configure lambda to send an invocation record to another service like SQS, SNS, lambda.
- List of services
- Amazon S3
- Amazon SNS
- Cloud Formation
- Cloud watch Logs
- Cloud watch events
- Aws code commit
- Aws config



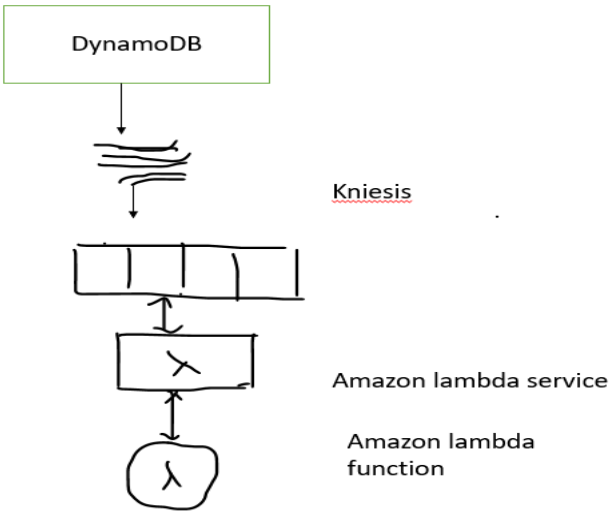
(c)Poll-Based

- The invocation model is designed to allow you to integrate with AWS stream and queue based service with no code or server management lambda will poll the following service on your behalf , retrieve records and invoke your function
- The following are supported service

Amazon kinesis

Amazon SQS

Amazon DynamoDB streams



Step by Step Procedure for S3 trigger by lambda and dynamo DB:

(1) Creating IAM Role

- Go to IAM service □□create a role
- Select the AWS Service and enable Lambda □□click on Next
- Select the role as AWS DynamoDBFullAccess □next
- Give the name to the role here -□create role
- Role is successfully created

(2) Creating lambda function

- Now go to the Lambda service ,and create the function
- Choose the option Author from scratch ,give function name and select the language as python 3.7
- In permissions □change default execution role □□select the use an existing role □□create function
- Function is successfully created.

- Click on code → copy the below code in code area -→ click on Deploy

```
import boto3
from uuid import uuid4
def lambda_handler(event, context):
    s3 = boto3.client("s3")
    dynamodb = boto3.resource('dynamodb')
    for record in event['Records']:
        bucket_name = record['s3']['bucket']['name']
        object_key = record['s3']['object']['key']
        size = record['s3']['object'].get('size', -1)
        event_name = record ['eventName']
        event_time = record['eventTime']
        dynamoTable = dynamodb.Table('newtable')
        dynamoTable.put_item(
            Item={'unique': str(uuid4()), 'Bucket': bucket_name,
'Object': object_key,'Size': size, 'Event': event_name,
'EventTime': event_time})
```

(3) Create S3 Bucket

- Go to S3 service and create bucket
- Give the bucket name globally unique
- Uncheck the block all public access, and acknowledge it → create bucket

(4) Create Dynamo

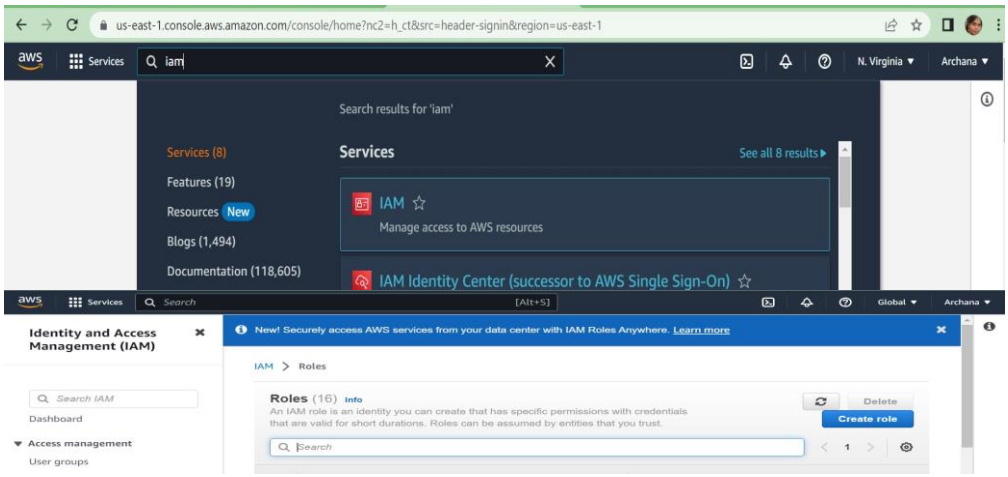
- Go to service DynamoDB and create the table
- Give the table as newtable and partition key as unique which we provided in the code □□create table
- Go lambda function □Add trigger -□□Select the service □□select the bucket □Select the All object create events
- Acknowledge it □add

(5) Upload file in S3 bucket

- Go to S3 and upload file in the bucket
- Here we have uploaded the file

(6) Check in dynamo DB about file details

- Come to DynamoDB here we can see tables in that click explore items ,the files of s3 has been displayed.



DOCKER

Docker is an open-source platform that enables developers to automate the deployment, scaling, and management of applications using **containerization**. Containers are lightweight, portable, and self-sufficient units that include everything needed to run a piece of software, including code, runtime, libraries, and dependencies.

Docker helps eliminate the "works on my machine" problem by ensuring consistency across different environments, from development to production.

Key Features of Docker

- **Lightweight**: Containers share the host OS kernel, making them more efficient than virtual machines (VMs).
- **Portability**: Containers can run on any system with Docker installed, regardless of the underlying OS.
- **Scalability**: Docker enables fast scaling of applications by running multiple container instances.
- **Isolation**: Containers ensure process isolation, enhancing security and stability.
- **Microservices Support**: Docker enables the development and deployment of microservices-based applications.

Docker Architecture

1. Docker Components

Docker follows a **client-server** architecture and consists of the following key components:

- **Docker Client**: A command-line interface (CLI) that interacts with the Docker daemon.
- **Docker Daemon (dockerd)**: Manages containers and handles container lifecycle operations.
- **Docker Images**: Read-only templates used to create containers.
- **Docker Containers**: Instances of Docker images that run applications.
- **Docker Registry (Docker Hub/Private Registry)**: A centralized repository to store and distribute Docker images.
- **Docker Compose**: A tool to define and manage multi-container applications.

2. Docker Workflow

1. **Build**: Developers create a Dockerfile to define an image.
2. **Push**: The image is uploaded to a registry like Docker Hub.
3. **Pull**: The image is downloaded onto the target system.
4. **Run**: Containers are instantiated from the image.

Docker in Cloud Computing

How Docker is Used in the Cloud?

Docker is widely used in cloud environments for deploying and managing containerized applications. Cloud providers such as **AWS, Azure, and Google Cloud** offer container orchestration services to manage Docker containers efficiently.

- **Container Orchestration**: Docker containers are managed using tools like **Kubernetes, Docker Swarm, or Amazon ECS** in cloud environments.
- **Multi-Cloud Compatibility**: Docker provides a consistent environment across different cloud platforms.
- **CI/CD Integration**: Docker is an essential component of **Continuous Integration and Deployment (CI/CD)** pipelines.
- **Resource Optimization**: Containers consume fewer resources compared to VMs, making cloud resource usage more efficient.

Docker Cloud Architecture

The typical cloud-based Docker architecture consists of:

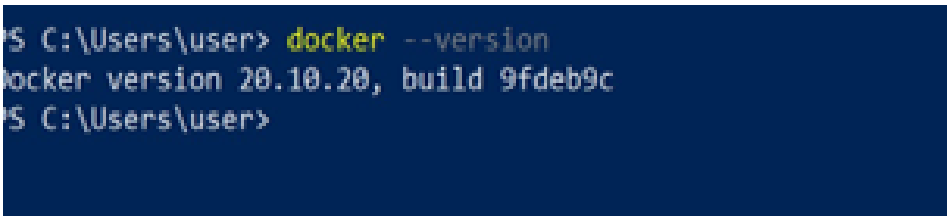
1. **Developer Environment**
   - Developers create and test Docker images on local machines.
2. **Container Registry**
   - Stores Docker images (Docker Hub, Amazon ECR, Google Container Registry).
3. **Cloud Infrastructure**
   - Containers run on cloud VMs, serverless platforms, or managed Kubernetes services.
4. **Networking & Security**
   - Load balancers, service discovery, and security policies ensure smooth operation.

```
+-----------------------+
|  Developer Environment |
| (Local Machine, IDE)   |
+-----------+------------+
            |
            v
+-----------------------+
|  Container Registry    |
| (Docker Hub, ECR, GCR) |
+-----------+------------+
            |
            v
+-----------------------+
| Cloud Infrastructure  |
| (AWS, Azure, GCP VMs) |
+-----------+------------+
            |
            v
+-----------------------+
| Networking & Security  |
| (Load Balancers, Firewalls) |
+-----------------------+
```

Docker Commands

1. **docker –version**

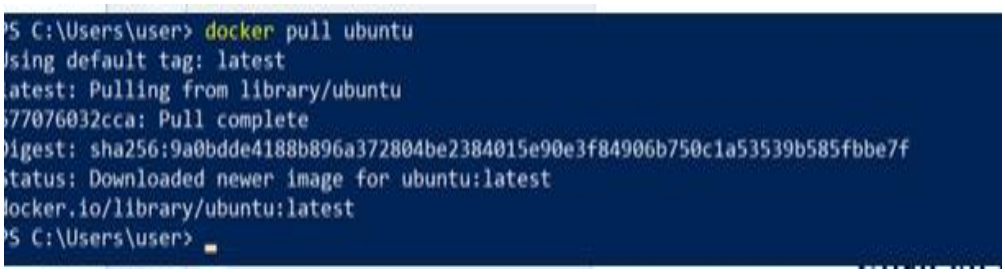
   This command is used to get the currently installed version of docker



2. **docker pull**

   Usage: docker pull <image name>

   This command is used to pull images from the **docker repository**(hub.docker.com



3. **docker run**

   Usage: docker run -it -d <image name>

   This command is used to create a container from an image

```
PS C:\Users\user> docker run --name myubuntu -it -d ubuntu
453c8123ec81cf89c7b22e080f707ee9b33971e73d2ecaad1d69cfc801c0ce6b
PS C:\Users\user>
```

4. **docker ps**

   This command is used to list the running containers

```
PS C:\Users\user> docker ps
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS          PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   43 seconds ago   Up 42 seconds             myubuntu
PS C:\Users\user>
```

5. **docker ps -a**

   This command is used to show all the running and exited containers

```
PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED            STATUS            PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   About a minute ago  Up About a minute          myubuntu
```

6. **docker stop**

   Usage: docker stop <container id>

   This command stops a running container

```
PS C:\Users\user> docker stop 453c8123ec81
453c8123ec81
PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS                      PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   3 minutes ago    Exited (137) 10 seconds ago            myubuntu
PS C:\Users\user>
```

7. **docker kill**

   Usage: docker kill <container id>

   This command kills the container by stopping its execution immediately. The difference between 'docker kill' and 'docker stop' is that 'docker stop' gives the container time to shutdown gracefully, in situations when it is taking too much time for getting the container to stop

8. **docker commit**

   Usage: docker commit <conatainer id> <username/imagename>

   This command creates a new image of an edited container on the local system

```
PS C:\Users\user> docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
ubuntu       latest    58db3edaf2be   4 weeks ago   77.8MB
PS C:\Users\user>
```

9. **docker build**

Usage: docker build <path to docker file>

This command is used to build an image from a specified docker file.

```
PS C:\Users\user\Downloads\fileshare-main\fileshare-main> docker build -t fileshare .
[+] Building 9.5s (4/10)
 => [1/5] FROM docker.io/library/node@sha256:50b76fc6dc5f03cb3d14c71b8564948aed2bc5124325f35830b2f3be21  5.0s
 => => sha256:5c8cfbf51e6e6869f1af2a1e7067b07fd6733036a333c9d29f743b0285e26037 2.10MB / 5.16MB           4.9s
 => => sha256:aa3a609d15798d35c1484072876b7d22a316e98de6a097de33b9dade6a689cd1 8.39MB / 10.88MB          4.9s
 => => sha256:f2f58072e9ed1aa1b0143341c5ee83815c00ce47548309fa240155067ab0e698 2.10MB / 55.04MB          4.9s
 => [internal] load build context                                                                        5.0s
 => => transferring context: 2.79MB                                                                      4.8s
```

10. **docker push**

Usage: docker push <username/image name>

This command is used to push an image to the docker hub repository

```
PS C:\Users\user\Downloads\fileshare-main\fileshare-main> docker push archanareddycse/fileshare1
Using default tag: latest
The push refers to repository [docker.io/archanareddycse/fileshare1]
f467781874d7: Pushed
b96c3920cda1: Pushed
1a044c6957a0: Pushed
efdadabceebf: Pushed
bd64d88b305d: Mounted from library/node
a5ba4d1f9d2f: Mounted from library/node
d29a2b8b4664: Mounted from library/node
5461a69cb3a7: Mounted from library/node
4ff8844d474a: Mounted from library/node
b77487480ddb: Mounted from library/node
cd247c0fb37b: Mounted from library/node
cfdd5c3bd77e: Mounted from library/node
870a241bfebd: Mounted from library/node
latest: digest: sha256:19aeeeaefdf9a20d4377ddb4b92c4ffa524c4bf6b93db18f279532c591790fa7 size: 3053
PS C:\Users\user\Downloads\fileshare-main\fileshare-main>
```

```
PS C:\Users\user> docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to
https://hub.docker.com to create one.
Username: archanareddycse
Password:
Login Succeeded
```

11. **docker rm**

Usage: docker rm <container id>

This command is used to delete a stopped container

```
PS C:\Users\user> docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED        STATUS                     PORTS     NAMES
453c8123ec81   ubuntu    "/bin/bash"   24 hours ago   Exited (137) 24 hours ago            myubuntu
PS C:\Users\user> docker rm 453c8123ec81
453c8123ec81
```

12. **docker rmi**

Usage: docker rmi <image-id>

This command is used to delete an image from local storage

```
PS C:\Users\user> docker rmi 58db3edaf2be
Untagged: ubuntu:latest
Untagged: ubuntu@sha256:9a0bdde4188b896a372804be2384015e90e3f84906b750c1a53539b585fbbe7f
Deleted: sha256:58db3edaf2be6e80f628796355b1bdeaf8bea1692b402f48b7e7b8d1ff100b02
Deleted: sha256:c5ff2d88f67954bdcf1cfdd46fe3d683858d69c2cadd6660812edfc83726c654
PS C:\Users\user>
```

<u>Step by step Procedure to Pull the image from Docker to EC2 Instance aand access it publicly.</u>

- Create the EC2 instance and connect with EC2 Console.
- In the opened Console of EC2, Type the following commands to pull the image from docker - sudo apt update / sudo apt-get update

- sudo apt install docker.io / sudo apt-get install docker .io
- sudo docker version
- sudo docker image ls (it shows the images list present in our Instance)

  Note: we don't have images in the instance because we didn't pulled the image from docker

- sudo docker pull scott2srikanth/fileshare_docker-fdp (pull the image)
- sudo docker image ls (shows the image pulled in the list)
- sudo docker run –d –p 3000:3000 scott2srikanth/fsdreactdemo

Note: 3000:3000 first is inbound values we change it but right isde 3000 value is docker bound values we cant change that, example we can give 3008:3000 -

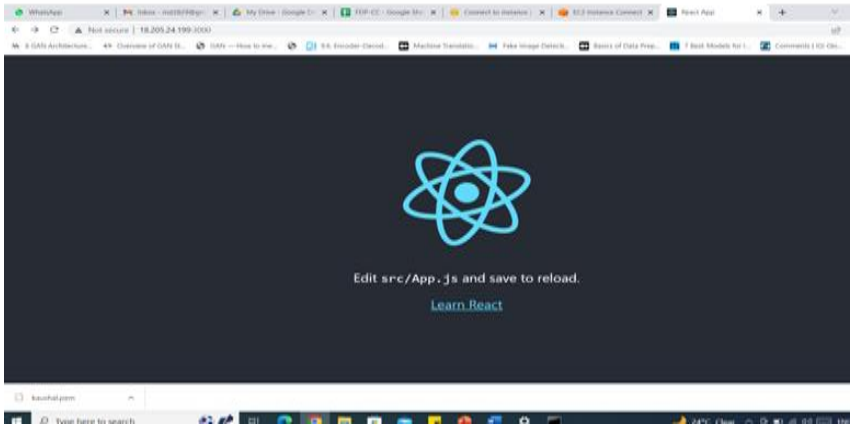
• After run command it shows the image downloaded. Now we can access it publicly, by copying the EC2 public IP address shown below of the console or EC2 dashboard.

• Copy the public ip and paste it on browser with inbound value.

• Example : http://3.12.123.4:3000 Note : if it is not opening check

(a) whether u have given https or http or

(b) Go to dashboard ,click on EC2 instance ,it shows security tab.. click on Edit inbound rules, click on Add rule .select All traffic and anywhere from IPV4 and save it and refresh the browser. Finally the pulled image will be displayed.



Conclusion

Docker has revolutionized modern application development by enabling **lightweight, portable, and scalable** containerized deployments. It plays a crucial role in cloud computing by offering **efficiency, flexibility, and seamless deployment** across cloud platforms.